SQL COMPLETE NOTES

From Beginner to Data Science & Placement Level

---

1. What is SQL?

SQL = Structured Query Language

SQL is used to:

- Store data
- Retrieve data
- Filter data
- Analyze data
- Update data
- Delete data
- Combine data from multiple tables
- Perform calculations
- Create and modify database structures

SQL is especially important for:

- Data Science
- Data Analytics
- Business Intelligence
- Software Engineering
- Backend Development
- Machine Learning data preparation
- Data Engineering

SQL is used with databases such as:

- MySQL
- PostgreSQL
- SQLite
- Oracle
- Microsoft SQL Server

SQLBolt teaches common SQL syntax that applies across relational databases, although individual database systems have additional features and differences.

---

2. What is a Database?

A **database** is an organized collection of data.

Example:

A college database may contain:

Students

Teachers

Courses

Marks

Attendance

Departments

A company database may contain:

Employees

Customers

Products

Orders

Payments

Departments

---

3. What is a Relational Database?

A relational database stores information in **tables**.

A table contains:

- Rows
- Columns

Example:

Students

id	name	department	cgpa
1	Anu	CSE	8.5
2	Ravi	ECE	7.8
3	Priya	CSE	9.1

Column

A column represents an attribute.

Examples:

name

department

cgpa

Row

A row represents one record.

Example:

1 | Anu | CSE | 8.5

---

4. Database Terminology

SQL term	Meaning
Database	Collection of related data
Table	Data organized into rows and columns
Row	One record
Column	One attribute
Primary Key	Uniquely identifies a row
Foreign Key	Connects two tables
Query	Request for data
Schema	Structure/design of database

---

5. SQL Command Categories

SQL commands can broadly be divided into:

DQL — Data Query Language

Used to retrieve data.

SELECT

DML — Data Manipulation Language

Used to modify data.

INSERT

UPDATE

DELETE

DDL — Data Definition Language

Used to define database structures.

CREATE

ALTER

DROP

TRUNCATE

DCL — Data Control Language

Used for permissions.

GRANT

REVOKE

TCL — Transaction Control Language

Used to manage transactions.

COMMIT

ROLLBACK

SAVEPOINT

For Data Science, the most frequently used areas are:

SELECT

WHERE

GROUP BY

HAVING

JOIN

CASE

Subqueries

CTEs

Window Functions

Aggregate Functions

---

6. SELECT — Most Important Beginner Concept

SELECT retrieves data from a table.

Syntax:

SELECT column_name

FROM table_name;

Example:

SELECT name

FROM Students;

Multiple columns:

SELECT name, cgpa

FROM Students;

All columns:

SELECT *

FROM Students;

* means all columns.

SQLBolt introduces SELECT as the fundamental way to retrieve data from a SQL database.

---

7. DISTINCT

DISTINCT removes duplicate values.

Example:

SELECT DISTINCT department

FROM Students;

Result:

CSE

ECE

ISE

ME

Without DISTINCT, repeated departments may appear multiple times.

---

8. WHERE

WHERE filters rows.

SELECT *

FROM Students

WHERE cgpa > 8;

Only students with CGPA greater than 8 are returned.

---

9. Comparison Operators

Operator	Meaning
=	Equal
<>	Not equal
!=	Not equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Example:

SELECT *

FROM Students

WHERE cgpa >= 8.5;

---

10. AND

Both conditions must be true.

```
SELECT *
FROM Students
WHERE department = 'CSE'
AND cgpa > 8;
```

Meaning:

Department must be CSE

AND

CGPA must be greater than 8

---

11. OR

At least one condition must be true.

```
SELECT *
FROM Students
WHERE department = 'CSE'
OR department = 'ISE';
```

---

12. NOT

Used to reverse a condition.

```
SELECT *
FROM Students
WHERE NOT department = 'CSE';
```

---

13. ORDER BY

Used to sort results.

Ascending:

```
SELECT *
FROM Students
```

ORDER BY cgpa ASC;

Descending:

SELECT *

FROM Students

ORDER BY cgpa DESC;

ASC = ascending

DESC = descending

Example:

SELECT name, salary

FROM Employees

ORDER BY salary DESC;

This displays the highest salary first.

---

14. LIMIT

Used to restrict the number of rows returned.

SELECT *

FROM Students

LIMIT 5;

Top 5 students by CGPA:

SELECT *

FROM Students

ORDER BY cgpa DESC

LIMIT 5;

Note: Some databases use different syntax for limiting rows.

---

15. LIKE

Used for pattern matching.

Names beginning with A:

SELECT *

FROM Students

WHERE name LIKE 'A%';

% means any number of characters.

Names ending with a:

WHERE name LIKE '%a';

Names containing "an":

WHERE name LIKE '%an%';

_ represents one character.

WHERE name LIKE 'A_i';

---

16. IN

Used when checking multiple possible values.

Instead of:

WHERE department = 'CSE'

OR department = 'ISE'

OR department = 'ECE'

Use:

WHERE department IN ('CSE', 'ISE', 'ECE');

---

17. BETWEEN

Used for a range.

SELECT *

FROM Employees

WHERE salary BETWEEN 40000 AND 60000;

For dates:

WHERE joining_date

BETWEEN '2026-01-01' AND '2026-06-30';

---

18. NULL

NULL means missing/unknown value.

It is NOT the same as:

0

''

'NULL'

To find NULL:

```
SELECT *
FROM Employees
WHERE salary IS NULL;
```

To find non-NULL:

```
SELECT *
FROM Employees
WHERE salary IS NOT NULL;
```

Never use:

```
WHERE salary = NULL
```

Use:

```
WHERE salary IS NULL
```

SQLBolt has a dedicated lesson on NULL because handling missing values correctly is important in SQL.

---

19. Aliases — AS

Aliases give temporary names to columns or tables.

```
SELECT name AS student_name
FROM Students;
```

Example:

```
SELECT salary * 12 AS annual_salary
FROM Employees;
```

Table alias:

```
SELECT s.name
```

FROM Students AS s;

Short form:

SELECT s.name

FROM Students s;

Aliases are extremely useful with JOINs.

---

20. Arithmetic Operations

SQL can perform calculations.

SELECT salary * 12

FROM Employees;

Operators:

+

-

*

/

%

Example:

SELECT price, quantity,

    price * quantity AS total

FROM Orders;

---

21. Aggregate Functions

Aggregate functions perform calculations on multiple rows.

The most important ones are:

COUNT()

SUM()

AVG()

MIN()

MAX()

---

22. COUNT()

Count rows:

```
SELECT COUNT(*)
FROM Students;
```

Count values:

```
SELECT COUNT(email)
FROM Students;
```

Count unique departments:

```
SELECT COUNT(DISTINCT department)
FROM Students;
```

---

23. SUM()

Calculate total.

```
SELECT SUM(salary)
FROM Employees;
```

---

24. AVG()

Calculate average.

```
SELECT AVG(salary)
FROM Employees;
```

---

25. MIN()

Find minimum.

```
SELECT MIN(salary)
FROM Employees;
```

---

26. MAX()

Find maximum.

SELECT MAX(salary)

FROM Employees;

---

27. GROUP BY — VERY IMPORTANT FOR DATA SCIENCE

GROUP BY creates groups of similar values.

Example:

SELECT department, COUNT(*)

FROM Employees

GROUP BY department;

Result:

department	count
IT	25
HR	10
Sales	15

Meaning:

Count employees separately for each department.

Another example:

SELECT department, AVG(salary)

FROM Employees

GROUP BY department;

This gives average salary by department.

---

28. HAVING

HAVING filters groups.

Example:

SELECT department, AVG(salary)

FROM Employees

GROUP BY department

HAVING AVG(salary) > 50000;

WHERE vs HAVING

WHERE filters **rows before grouping**.

HAVING filters **groups after grouping**.

Remember:

WHERE → rows

HAVING → groups

---

29. JOINs — VERY IMPORTANT ⭐⭐⭐⭐⭐

Real-world databases usually contain multiple tables.

Example:

Students

student_id	name
1	Anu
2	Ravi

Applications

application_id	student_id	company
101	1	Google
102	2	Microsoft

We need JOIN to combine them.

---

30. INNER JOIN

Returns matching records from both tables.

SELECT s.name, a.company

FROM Students s

INNER JOIN Applications a

ON s.student_id = a.student_id;

Think:

Students

+

Applications

↓

Matching records

---

31. LEFT JOIN

Returns all records from the left table and matching records from the right.

SELECT s.name, a.company

FROM Students s

LEFT JOIN Applications a

ON s.student_id = a.student_id;

Useful question:

Show ALL students, including students who haven't applied anywhere.

---

32. RIGHT JOIN

Returns all records from the right table and matching records from the left.

SELECT s.name, a.company

FROM Students s

RIGHT JOIN Applications a

ON s.student_id = a.student_id;

---

33. FULL OUTER JOIN

Returns records from both sides, matching where possible.

SELECT *

FROM Students s

FULL OUTER JOIN Applications a

ON s.student_id = a.student_id;

Not all database systems support this syntax directly.

---

34. SELF JOIN

A table can be joined with itself.

Example:

Employee table:

employee_id

name

manager_id

Query:

```
SELECT e.name AS employee,
    m.name AS manager
FROM Employees e
LEFT JOIN Employees m
ON e.manager_id = m.employee_id;
```

---

35. CROSS JOIN

Produces combinations of every row from one table with every row from another.

```
SELECT *
FROM Students
CROSS JOIN Courses;
```

Be careful: it can produce a very large result.

---

36. CASE

CASE is SQL's conditional logic.

Example:

```
SELECT name,
    cgpa,
    CASE
        WHEN cgpa >= 9 THEN 'Excellent'
```

```
        WHEN cgpa >= 8 THEN 'Good'
        WHEN cgpa >= 7 THEN 'Average'
        ELSE 'Needs Improvement'
    END AS performance
FROM Students;
```

This is extremely useful in Data Analytics.

---

37. String Functions

Common functions:

UPPER()

LOWER()

LENGTH()

TRIM()

CONCAT()

SUBSTRING()

REPLACE()

Example:

```
SELECT UPPER(name)
FROM Students;
```

Example:

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM Employees;
```

Exact function names can vary between database systems.

---

38. Date Functions

Dates are extremely important in Data Science.

Typical operations include:

Current date

Date difference

Extract year

Extract month

Extract day

Date addition/subtraction

Examples vary by database.

PostgreSQL:

```
SELECT EXTRACT(YEAR FROM joining_date)
FROM Employees;
```

MySQL:

```
SELECT YEAR(joining_date)
FROM Employees;
```

---

39. Subqueries

A subquery is a query inside another query.

Example:

Find employees earning more than the average salary:

```
SELECT *
FROM Employees
WHERE salary >
(
   SELECT AVG(salary)
   FROM Employees
);
```

Inner query:

```
SELECT AVG(salary)
FROM Employees;
```

Outer query:

```
SELECT *
FROM Employees
```

```
WHERE salary > average;
```

---

40. IN with Subquery

Find students who belong to departments located in a particular region:

```
SELECT *
FROM Students
WHERE department_id IN
(
   SELECT department_id
   FROM Departments
   WHERE region = 'South'
);
```

---

41. EXISTS

Checks whether a subquery returns at least one record.

```
SELECT *
FROM Students s
WHERE EXISTS
(
   SELECT 1
   FROM Applications a
   WHERE a.student_id = s.student_id
);
```

Meaning:

Find students who have at least one application.

---

42. CTE — Common Table Expression

CTEs make complicated queries easier to read.

Syntax:

```
WITH high_salary AS
(
   SELECT *
   FROM Employees
   WHERE salary > 60000
)
SELECT *
FROM high_salary;
```

Think:

```
WITH temporary_result AS (...)
SELECT ...
```

CTEs are heavily used in analytics and modern SQL.

---

43. Multiple CTEs

```
WITH employee_data AS
(
   SELECT *
   FROM Employees
),
high_salary AS
(
   SELECT *
   FROM employee_data
   WHERE salary > 60000
)
SELECT *
FROM high_salary;
```

---

44. UNION

Combines results from two queries.

SELECT name

FROM Students_CSE

UNION

SELECT name

FROM Students_ISE;

UNION removes duplicates.

---

45. UNION ALL

SELECT name

FROM Students_CSE

UNION ALL

SELECT name

FROM Students_ISE;

UNION ALL keeps duplicates.

Usually:

UNION → remove duplicates

UNION ALL → keep duplicates

---

46. INTERSECT

Returns values present in both queries.

SELECT student_id

FROM Python_Students

INTERSECT

SELECT student_id

FROM SQL_Students;

Meaning:

Students who know both Python and SQL.

Support varies by database.

---

47. EXCEPT

Returns values from the first query that aren't in the second.

SELECT student_id

FROM Students

EXCEPT

SELECT student_id

FROM Applications;

Meaning:

Students who have not applied.

Database syntax/support can vary.

---

48. Window Functions — VERY IMPORTANT ⭐⭐⭐⭐⭐

Window functions are one of the most important advanced SQL concepts for Data Science/Data Analytics.

They perform calculations across related rows **without collapsing the rows like GROUP BY does**.

Common functions:

ROW_NUMBER()

RANK()

DENSE_RANK()

LAG()

LEAD()

SUM() OVER()

AVG() OVER()

---

49. ROW_NUMBER()

Example:

```
SELECT name,
    salary,
    ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
FROM Employees;
```

This assigns:

1

2

3

4

...

based on salary ranking.

---

50. RANK()

```
SELECT name,
    salary,
    RANK() OVER (ORDER BY salary DESC) AS rank
FROM Employees;
```

If two employees have the same salary:

Salary	Rank
70000	1
70000	1
65000	3

There is a gap.

---

51. DENSE_RANK()

```
SELECT name,
    salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS rank
FROM Employees;
```

Result:

70000 → 1

70000 → 1

65000 → 2

No gap.

---

52. PARTITION BY

Partitioning creates groups for window calculations.

Example:

```
SELECT name,
    department,
    salary,
    RANK() OVER
    (
       PARTITION BY department
       ORDER BY salary DESC
    ) AS dept_rank
FROM Employees;
```

Meaning:

Rank employees separately inside each department.

This is extremely useful for Data Analytics.

---

53. LAG()

Looks at a previous row.

Example:

```
SELECT month,
    sales,
    LAG(sales) OVER (ORDER BY month) AS previous_sales
FROM Monthly_Sales;
```

Useful for:

- Month-over-month analysis
- Previous transaction
- Previous salary
- Time-series analysis

---

54. LEAD()

Looks at the next row.

```
SELECT month,
    sales,
    LEAD(sales) OVER (ORDER BY month) AS next_sales
FROM Monthly_Sales;
```

---

55. Running Total

```
SELECT date,
    sales,
    SUM(sales) OVER
    (
       ORDER BY date
    ) AS running_total
FROM Sales;
```

Useful for:

- Revenue tracking
- Cumulative sales
- Financial analysis
- Time-series analysis

---

56. SQL Query Execution Order

This is extremely important for interviews.

Although we write:

SELECT

FROM

WHERE

GROUP BY

HAVING

ORDER BY

LIMIT

SQL logically processes the query approximately as:

FROM

↓

WHERE

↓

GROUP BY

↓

HAVING

↓

SELECT

↓

ORDER BY

↓

LIMIT

Example:

SELECT department, AVG(salary)

FROM Employees

WHERE salary > 30000

GROUP BY department

HAVING AVG(salary) > 50000

ORDER BY AVG(salary) DESC;

Understanding execution order helps you understand why aliases and aggregate conditions behave differently in different clauses.

SQLBolt includes a dedicated lesson on query execution order.

---

57. INSERT

Adds a new record.

INSERT INTO Students

(name, department, cgpa)

VALUES

('Vaishnavi', 'CSE', 8.6);

---

58. UPDATE

Modifies existing records.

UPDATE Students

SET cgpa = 8.8

WHERE name = 'Vaishnavi';

IMPORTANT:

Always be careful with UPDATE.

This is dangerous:

UPDATE Students

SET cgpa = 8.8;

It updates every row.

---

59. DELETE

Deletes rows.

```
DELETE FROM Students
WHERE id = 5;
```

Again, be careful.

This:

```
DELETE FROM Students;
```

deletes all rows.

---

60. CREATE TABLE

Creates a table.

```
CREATE TABLE Students
(
   student_id INT,
   name VARCHAR(100),
   department VARCHAR(50),
   cgpa DECIMAL(3,2)
);
```

---

61. Primary Key

A primary key uniquely identifies a row.

```
CREATE TABLE Students
(
   student_id INT PRIMARY KEY,
   name VARCHAR(100),
   cgpa DECIMAL(3,2)
);
```

Example:

1 → Anu

2 → Ravi

3 → Priya

Each student_id must be unique.

---

62. Foreign Key

Connects two tables.

```
CREATE TABLE Applications
(
   application_id INT PRIMARY KEY,
   student_id INT,
   company VARCHAR(100),

   FOREIGN KEY (student_id)
   REFERENCES Students(student_id)
);
```

Relationship:

```
Students
   │
   │ student_id
  ↓
Applications
```

---

63. Important Constraints

Common constraints:

PRIMARY KEY

FOREIGN KEY

NOT NULL

UNIQUE

CHECK

DEFAULT

Example:

```
CREATE TABLE Employees
(
   id INT PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   email VARCHAR(150) UNIQUE,
   salary DECIMAL(10,2) CHECK (salary > 0),
   country VARCHAR(50) DEFAULT 'India'
);
```

---

64. ALTER TABLE

Used to modify a table structure.

Examples vary slightly by database.

```
ALTER TABLE Employees
ADD COLUMN age INT;
```

Rename:

```
ALTER TABLE Employees
RENAME COLUMN name TO employee_name;
```

---

65. DROP TABLE

Deletes the entire table structure and its data.

```
DROP TABLE Students;
```

Be careful.

---

66. DELETE vs TRUNCATE vs DROP

DELETE

```
DELETE FROM Students
```

WHERE id = 5;

Removes selected rows.

TRUNCATE

TRUNCATE TABLE Students;

Removes all rows while keeping the table structure.

DROP

DROP TABLE Students;

Removes the table itself.

---

67. Normalization

Normalization is a database design concept.

Purpose:

- Reduce duplicate data
- Improve consistency
- Make database structure cleaner

Important forms:

1NF

2NF

3NF

BCNF

For placements, understand the basic idea and examples.

---

68. Indexes

Indexes make data retrieval faster.

Example:

CREATE INDEX idx_employee_name

ON Employees(name);

Without indexes, databases may need to scan many rows.

But indexes also have costs:

- Additional storage
- Slower INSERT/UPDATE/DELETE in some cases
- Maintenance overhead

So indexes should be used thoughtfully.

---

69. Transactions

A transaction is a group of database operations treated as one logical unit.

Example:

Transfer ₹1000

↓

Remove ₹1000 from Account A

↓

Add ₹1000 to Account B

Both should succeed together.

Important commands:

BEGIN;

COMMIT;

ROLLBACK;

---

70. ACID Properties

Very important interview concept.

A — Atomicity

All operations happen or none happen.

C — Consistency

Database remains valid.

I — Isolation

Transactions don't improperly interfere with one another.

D — Durability

Committed data remains saved.

Remember:

A → Atomicity

C → Consistency

I → Isolation

D → Durability

---

71. SQL for Data Science

Now comes the most important part for your goal.

A Data Scientist usually doesn't just retrieve:

SELECT *

FROM table;

They use SQL to:

Extract data

↓

Clean data

↓

Transform data

↓

Aggregate data

↓

Create features

↓

Analyze patterns

↓

Prepare ML datasets

---

72. Data Cleaning with SQL

Find NULL values:

```
SELECT *
FROM Customers
WHERE age IS NULL;
```

Replace NULL:

```
COALESCE(age, 0)
```

Example:

```
SELECT name,
    COALESCE(age, 0) AS age
FROM Customers;
```

---

73. Remove Duplicate Results

```
SELECT DISTINCT city
FROM Customers;
```

For actual duplicate-row cleaning, the approach depends on your database and table design.

---

74. CASE for Feature Creation

Example:

```
SELECT customer_id,
    age,
    CASE
        WHEN age < 25 THEN 'Young'
        WHEN age < 50 THEN 'Adult'
        ELSE 'Senior'
    END AS age_group
FROM Customers;
```

This creates a new analytical feature.

---

75. Data Aggregation

Example:

```
SELECT city,
    COUNT(*) AS customers,
    AVG(income) AS average_income
FROM Customers
GROUP BY city;
```

This can produce a dataset ready for analysis.

---

76. Percentage Calculation

Example:

```
SELECT
  SUM(CASE WHEN status = 'Selected' THEN 1 ELSE 0 END) * 100.0
  / COUNT(*) AS selection_rate
FROM Applications;
```

Useful for:

- Conversion rate
- Selection rate
- Churn rate
- Error rate
- Success rate

---

77. Conditional Aggregation

Example:

```
SELECT
  COUNT(*) AS total_employees,

  SUM(CASE
    WHEN department = 'IT'
    THEN 1 ELSE 0
```

```
    END) AS it_employees

FROM Employees;
```

Very useful in analytics.

---

78. Data Science Example — Customer Analysis

Suppose:

Customers

customer_id

age

city

income

Orders

order_id

customer_id

order_date

amount

Question:

Find total spending for each customer.

```
SELECT
    c.customer_id,
    c.city,
    SUM(o.amount) AS total_spending
FROM Customers c
JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY
    c.customer_id,
    c.city;
```

This is exactly the type of SQL thinking used in real analytics.

---

79. SQL + Python

For Data Science, SQL and Python work together.

Typical workflow:

Database

↓

SQL

↓

Extract data

↓

Python

↓

Pandas

↓

EDA

↓

Visualization

↓

Machine Learning

Example Python:

```
import pandas as pd
import sqlite3

connection = sqlite3.connect("company.db")

df = pd.read_sql_query(
    """
    SELECT department, AVG(salary) AS avg_salary
```

```
    FROM Employees
    GROUP BY department
    """,
    connection
)

print(df)
```

---

80. SQL + Pandas

SQL does:

Filtering

Joining

Aggregation

Transformation

Pandas then does:

EDA

Visualization

Feature engineering

Machine learning

A Data Scientist should understand both.

---

81. Important SQL Concepts for Data Science

Your priority should be:

⭐⭐⭐⭐⭐ **MUST KNOW**

SELECT

WHERE

ORDER BY

GROUP BY

HAVING

JOIN

CASE

Aggregate functions

NULL handling

Subqueries

CTEs

Window functions

Date operations

String operations

⭐ ⭐ ⭐ ⭐ **SHOULD KNOW**

Primary keys

Foreign keys

Indexes

Constraints

Transactions

Normalization

Views

UNION

EXISTS

⭐ ⭐ ⭐ **ADVANCED**

Query optimization

Execution plans

Partitioning

Advanced window functions

Stored procedures

Triggers

Database performance

---

82. SQL Interview Questions You Must Know

Beginner

1. What is SQL?
2. What is a database?
3. What is a table?
4. What is a primary key?
5. What is a foreign key?
6. What is NULL?
7. What is DISTINCT?
8. What is WHERE?
9. What is ORDER BY?
10. What is GROUP BY?

Intermediate

11. WHERE vs HAVING?
12. DELETE vs TRUNCATE?
13. UNION vs UNION ALL?
14. INNER JOIN vs LEFT JOIN?
15. What is a subquery?
16. What is a CTE?
17. What is an index?
18. What is normalization?
19. What are aggregate functions?
20. What are constraints?

Advanced

21. RANK vs DENSE_RANK?
22. ROW_NUMBER vs RANK?
23. What are window functions?
24. What is PARTITION BY?
25. What is a transaction?
26. What is ACID?

27. How do you find duplicate records?

28. How do you find the second-highest salary?

29. How do you calculate running totals?

30. How do you find top N records per group?

---

83. Classic Interview Query — Second Highest Salary

One approach:

```
SELECT MAX(salary) AS second_highest
FROM Employees
WHERE salary <
(
   SELECT MAX(salary)
   FROM Employees
);
```

Another approach can use ranking/window functions.

---

84. Find Duplicate Values

```
SELECT email,
      COUNT(*) AS count
FROM Employees
GROUP BY email
HAVING COUNT(*) > 1;
```

---

85. Find Top 3 Salaries

```
SELECT *
FROM Employees
ORDER BY salary DESC
LIMIT 3;
```

For the top 3 **within each department**, use a window function:

```
WITH ranked AS
(
   SELECT *,
          DENSE_RANK() OVER
          (
             PARTITION BY department
             ORDER BY salary DESC
          ) AS rnk
   FROM Employees
)
SELECT *
FROM ranked
WHERE rnk <= 3;
```

This is an important real-world pattern.

---

86. SQL Learning Roadmap

Follow this order:

LEVEL 1

↓

Database basics

↓

SELECT

↓

WHERE

↓

DISTINCT

↓

ORDER BY

↓

LIMIT

↓

LIKE

↓

IN

↓

BETWEEN

LEVEL 2

↓

Aggregate functions

↓

GROUP BY

↓

HAVING

↓

CASE

↓

NULL

↓

String functions

↓

Date functions

LEVEL 3

↓

INNER JOIN

↓

LEFT JOIN

↓

RIGHT JOIN

↓

SELF JOIN

↓

Multiple-table queries

LEVEL 4

↓

Subqueries

↓

EXISTS

↓

CTEs

↓

UNION

↓

UNION ALL

↓

INTERSECT

↓

EXCEPT

LEVEL 5

↓

Window functions

↓

ROW_NUMBER

↓

RANK

↓

DENSE_RANK

↓

LAG

↓

LEAD

↓

Running totals

↓

PARTITION BY

LEVEL 6

↓

Database design

↓

Primary keys

↓

Foreign keys

↓

Constraints

↓

Normalization

↓

Indexes

↓

Transactions

↓

ACID

LEVEL 7

↓

Data Science SQL

↓

Data cleaning

↓

Feature creation

↓

Aggregation

↓

Time-series analysis

↓

Analytics queries

↓

SQL + Python/Pandas

---

87. SQLBolt Study Plan

Use SQLBolt alongside these notes.

SQLBolt currently organizes its core lessons approximately as:

1. Introduction
2. SELECT queries
3. Constraints/filtering
4. Filtering and sorting
5. SELECT review
6. Multi-table JOINs
7. OUTER JOINs
8. NULLs
9. Expressions

10. Aggregates
11. More aggregates
12. Query execution order
13. INSERT
14. UPDATE
15. DELETE
16. CREATE TABLE
17. ALTER TABLE
18. DROP TABLE

It also provides additional topics including:

- Subqueries
- UNION
- INTERSECT
- EXCEPT

SQLBolt

SQLBolt — Interactive SQL Lessons

Use this method:

Read my notes

↓

Study the SQLBolt lesson

↓

Complete the SQLBolt exercise

↓

Write 5 queries yourself

↓

Solve interview questions

↓

Use SQL on a real dataset

---

88. The Most Important SQL Skill

Don't memorize SQL queries.

Learn to translate:

Question → SQL logic

For example:

"Find the average salary of CSE employees."

Think:

What data?

→ salary

Which rows?

→ CSE

What calculation?

→ AVG

SQL:

```
SELECT AVG(salary)
FROM Employees
WHERE department = 'CSE';
```

Another:

"Find departments where average salary is greater than 60,000."

Think:

Group → department

Calculate → AVG(salary)

Filter groups → HAVING

```
SELECT department,
    AVG(salary) AS avg_salary
FROM Employees
```

GROUP BY department

HAVING AVG(salary) > 60000;

That **question → logic → query** skill is what you should develop for Data Science and interviews.

---

89. Your SQL Practice Rule

Every day:

20 minutes

Learn one concept.

30 minutes

Solve SQL problems.

10 minutes

Write your own query without looking at the answer.

10 minutes

Explain the query in words.

For example:

SELECT department, AVG(salary)

FROM Employees

GROUP BY department

HAVING AVG(salary) > 50000;

You should be able to say:

"First I group employees by department, calculate the average salary for each department, and then keep only departments whose average salary is above 50,000."

If you can explain your query like that, you actually understand SQL.

---

90. Final SQL Cheat Sheet

SELECT → retrieve data

DISTINCT → remove duplicate results

WHERE → filter rows

ORDER BY → sort

LIMIT → limit rows

LIKE → pattern matching

IN → match multiple values

BETWEEN → range

IS NULL → find missing values

COUNT() → count

SUM() → total

AVG() → average

MIN() → minimum

MAX() → maximum

GROUP BY → create groups

HAVING → filter groups

INNER JOIN → matching records

LEFT JOIN → all left + matching right

RIGHT JOIN → all right + matching left

FULL JOIN → all records from both sides

CASE → conditional logic

COALESCE() → handle NULL/default values

Subquery → query inside query

CTE → temporary named query

EXISTS → check whether rows exist

UNION → combine + remove duplicates

UNION ALL → combine + keep duplicates

ROW_NUMBER() → sequential number

RANK() → ranking with gaps

DENSE_RANK() → ranking without gaps

LAG() → previous row

LEAD() → next row

PARTITION BY → separate window groups

INSERT → add data

UPDATE → modify data

DELETE → remove data

CREATE → create structure

ALTER → modify structure

DROP → remove structure

PRIMARY KEY → unique row identifier

FOREIGN KEY → table relationship

INDEX → faster lookup

TRANSACTION → group of operations

ACID → transaction properties

Your target

For **Data Science**, don't stop after basic SELECT queries.

Your real target should be:

SQL → Advanced SQL → SQL + Pandas → Data Cleaning → EDA → Statistics → Visualization → Machine Learning